MyCodeSchool

L1 · Containerizare

MySQL în Docker Compose → backend-ul tău într-o imagine → pe Docker Hub → în același compose

Azi nu scrii cod de aplicație. Pornești o bază de date, iei un API pe care nu l-ai scris tu, îl împachetezi într-o imagine, îl urci pe Docker Hub și îl pornești lângă baza lui. Exact asta face un DevOps: aplicația vine de la alții, iar treaba ta e să meargă.

Fiecare pas are un criteriu de gata. Nu treci mai departe până nu îl vezi cu ochii tăi.

Ce trebuie să știi

- **Imaginea** = aplicația împachetată, cu tot ce îi trebuie ca să ruleze. Nu se schimbă.
- **Containerul** = o imagine pornită. Ca în C#: imaginea e clasa, containerul e obiectul.
- **Docker Compose** = un fișier în care scrii ce containere vrei, iar Docker le pornește pe toate.
- **Dockerfile** = rețeta din care se construiește imaginea.
- **Docker Hub** = locul de unde descarci imagini (ca MySQL) și unde le urci pe ale tale, ca să le poată porni oricine, de pe orice calculator.

Comenzile de azi

Comanda	Ce face
`docker compose up -d`	pornește tot ce scrie în `docker-compose.yaml`, în fundal
`docker compose ps`	arată containerele pornite
`docker compose logs NUME`	arată ce a scris containerul
`docker compose down`	oprește și șterge containerele; datele rămân
`docker build -t NUME:VERSIUNE .`	construiește o imagine din `Dockerfile`
`docker login`	te conectează la contul tău de Docker Hub
`docker push NUME:VERSIUNE`	urcă imaginea pe Docker Hub

Pregătire

1. Docker Desktop pornit. În *Settings* → *Resources* → *WSL integration*, Ubuntu bifat.
2. Cont pe hub.docker.com. Numele contului tău îl scrii în loc de `CONTUL-TAU` peste tot în fișa asta.
3. Lucrezi în terminalul **Ubuntu (WSL)**, în repo-ul clonat în `~`, nu în `/mnt/c/...`.

```
cd ~/devops-initiere
git pull
docker run hello-world
```

> **CRITERIU DE GATA**
>
> Apare `Hello from Docker!`

Partea 1 — MySQL în Docker Compose

MySQL nu îl construiești tu: e o imagine gata făcută, de pe Docker Hub. În rădăcina repo-ului creezi fișierul `docker-compose.yaml`:

```
services:
  mysql:
    image: mysql:8.4
    environment:
      MYSQL_ROOT_PASSWORD: root
    ports:
      - "3306:3306"
    volumes:
      - date-mysql:/var/lib/mysql
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-proot"]
      interval: 5s
      retries: 12

volumes:
  date-mysql:
```

Linia	Ce înseamnă
`mysql:`	numele serviciului; după numele ăsta îl găsesc celelalte containere
`image: mysql:8.4`	imaginea de pe Docker Hub și versiunea ei
`environment`	variabile date containerului; aici, parola lui `root`
`ports: "3306:3306"`	portul de pe calculatorul tău : portul din container
`volumes`	datele MySQL stau în volumul `date-mysql`, nu în container, deci supraviețuiesc lui
`healthcheck`	Docker întreabă MySQL la 5 secunde dacă e gata

```
docker compose up -d
docker compose ps
```

> **CRITERIU DE GATA**
>
> După 10–20 de secunde, `docker compose ps` arată `mysql` cu `(healthy)`.

Intri în MySQL, chiar în container:

```
docker compose exec mysql mysql -uroot -proot
```

```
CREATE DATABASE test_ana;
SHOW DATABASES;
exit
```

Apoi verifici că datele stau în volum, nu în container:

1. `docker compose down`, apoi `docker compose up -d`: intri din nou în MySQL, iar `SHOW DATABASES;` încă arată `test_ana`.
2. `docker compose down -v` șterge **și** volumul. După `docker compose up -d`, `test_ana` nu mai există.

> **CRITERIU DE GATA**
>
> Ai văzut `test_ana` supraviețuind lui `down` și dispărând după `down -v`.

Partea 2 — Containerizează backend-ul

Backend-ul e `proiecte/one-model-api-csharp`, un API ASP.NET pentru mașini. În folderul lui creezi fișierul `Dockerfile` (fără extensie), cu exact conținutul ăsta:

```
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src
COPY *.csproj ./
RUN dotnet restore
COPY . .
RUN dotnet publish -c Release -o /app/publish

FROM mcr.microsoft.com/dotnet/aspnet:8.0
WORKDIR /app
COPY --from=build /app/publish .
EXPOSE 8080
ENTRYPOINT ["dotnet", "OneModelApi.dll"]
```

Are două etape:

- **build**: o imagine cu SDK-ul .NET descarcă pachetele NuGet și face `dotnet publish`, adică exact ce face Visual Studio când compilezi;
- **runtime**: o imagine mică, doar cu ce trebuie ca să ruleze un API, primește **doar** rezultatul publicării și pornește `OneModelApi.dll` pe portul 8080.

Lângă el, fișierul `.dockerignore`, cu ce **nu** trimiți la build:

```
bin/
obj/
.vs/
```

> **ATENȚIE PE WINDOWS**
>
> Notepad salvează `Dockerfile.txt`, iar Docker nu-l găsește. Scrie fișierul cu `nano Dockerfile` în WSL sau din VS Code.

Build

```
cd proiecte/one-model-api-csharp
docker build -t CONTUL-TAU/one-model-api:1.0 .
```

`-t` dă imaginii nume și versiune. Numele începe cu contul tău, ca Docker Hub să știe unde s-o pună. Punctul de la final = „folderul ăsta e cel pe care îl trimiți la build”. Prima dată durează câteva minute.

> **CRITERIU DE GATA**
>
> ```
> docker images CONTUL-TAU/one-model-api
> ```
>
> Apare imaginea cu tag-ul `1.0`.

Push

```
docker login
docker push CONTUL-TAU/one-model-api:1.0
```

> **CRITERIU DE GATA**
>
> Pe `hub.docker.com`, la contul tău, apare repository-ul `one-model-api` cu tag-ul `1.0`.

Partea 3 — Backend-ul în Docker Compose

Te întorci în rădăcina repo-ului și adaugi al doilea serviciu în `docker-compose.yaml`, sub `mysql`:

```
  one-model:
    image: CONTUL-TAU/one-model-api:1.0
    ports:
      - "8102:8080"
    environment:
      ConnectionStrings__DefaultConnection: "Server=mysql;Database=cars_asp_db;User=root;Password=root"
    depends_on:
      mysql:
        condition: service_healthy
```

Linia	Ce înseamnă
`image: CONTUL-TAU/...`	imaginea ta, de pe Docker Hub, la fel cum `mysql` vine de acolo
`"8102:8080"`	aplicația ascultă pe 8080 în container; tu o vezi pe 8102
`ConnectionStrings__DefaultConnection`	același lucru cu `"ConnectionStrings": { "DefaultConnection": ... }` din `appsettings.json` ; cele două liniuțe țin locul acoladei
`Server=mysql`	adresa bazei e numele serviciului, nu `localhost` : în container, `localhost` e chiar containerul aplicației
`depends_on` + `service_healthy`	aplicația pornește abia după ce MySQL e `(healthy)`

```
cd ~/devops-initiere
docker compose up -d
docker compose ps
docker compose logs one-model
```

> **CRITERIU DE GATA**
>
> În loguri apare `EF Core migrations applied successfully.` : aplicația și-a făcut singură tabelele în MySQL. În browser, `http://localhost:8102/swagger` se deschide.

În Swagger, adaugă o mașină cu `POST /api/cars` :

```
{ "brand": "Dacia", "model": "Logan", "color": "alb", "size": 5,
  "year": 2020, "price": 8000, "mileage": 90000,
  "fuelType": "benzina", "transmission": "manuala", "available": true }
```

Apoi `docker compose down` și `docker compose up -d` .

> **CRITERIU DE GATA**
>
> După `down` și `up` , `GET /api/cars` încă îți arată Dacia ta: datele stau în volumul lui MySQL.

Când nu merge

Mai întâi te uiți la `docker compose ps -a` . Dacă la un serviciu scrie `Exited` , aplicația s-a oprit singură, iar `docker compose logs NUME` îți spune de ce.

Vezi	De ce	Ce faci
`ConnectionStrings:DefaultConnection is missing`	Lipsește variabila sau numele e scris greșit (o liniuță în loc de două)	Corectezi `docker-compose.yaml` , apoi `docker compose up -d`
`Unable to connect to any of the specified MySQL hosts`	În `Server=` scrie `localhost` sau alt nume decât `mysql`	La fel: corectezi, apoi `docker compose up -d`
`failed to read dockerfile`	Fișierul s-a salvat cu alt nume	`ls` în folder, apoi `mv Dockerfile.txt Dockerfile`
`push access denied`	Nu ești logată sau numele imaginii nu începe cu contul tău	`docker login` , verifici numele, din nou `build` și `push`
`must be lowercase`	Numele imaginii are litere mari	Scrii numele doar cu litere mici
`failed to resolve reference ... not found`	Numele din `image:` nu e la fel cu cel urcat	Copiezi numele exact de pe Docker Hub
`port is already allocated`	Alt program ține deja portul (de exemplu un MySQL instalat pe Windows, pe 3306)	Schimbi portul din stânga, de exemplu `"3307:3306"`
`no configuration file provided`	Nu ești în folderul cu `docker-compose.yaml`	`cd ~/devops-initiere`

Exercițiul — singură, pe cars-users

`proiecte/cars-users-api-csharp` e un API cu utilizatori și mașinile lor. Faci pentru el Partea 2 și Partea 3: `Dockerfile` , `.dockerignore` , build, push, apoi îl adaugi ca al **treilea** serviciu în același `docker-compose.yaml` , lângă `mysql` și `one-model` .

	cars-users
Fișierul pornit	`OneToManyApi.dll`
Imaginea	`CONTUL-TAU/cars-users-api:1.0`
Port	8080 în container, 8103 la tine
Baza de date	`one_to_many_db`, în **același** MySQL

> **CRITERIU DE GATA**
>
> `docker compose ps` arată 3 servicii. `localhost:8102/swagger` și `localhost:8103/swagger` se deschid în același timp, iar `GET /api/users` întoarce 2 utilizatori, fiecare cu mașinile lui.

La final

```
docker compose down
```

Apoi comiți `docker-compose.yaml`, cele două `Dockerfile` și `.dockerignore`, plus notițele în `notite/L1-containerizare.txt`, scrise cu cuvintele tale:

```
git add .
git commit -m "L1 containerizare"
git push
```

> **CRITERIU DE GATA**
>
> Pe GitHub se văd `docker-compose.yaml` în rădăcină și `Dockerfile` + `.dockerignore` în `one-model-api-csharp` și `cars-users-api-csharp`. Pe Docker Hub, la contul tău, se văd cele două imagini.